

Backend Architecture & Tech Stack

This document outlines the technical architecture, recommended technology stack, database schema and API design for the SkillConnect JK platform. The goal is to support the functional requirements defined in the PRD, meet non-functional constraints (security, scalability, offline support) and allow easy integration with banks, training providers and third-party services.

1. Architectural Overview

The platform will adopt a **modular, API-driven architecture** with a clear separation of concerns between user experience (front end), business logic (application layer), data persistence (database) and external integrations. At MVP stage, a **monolithic service** built on a scalable framework is sufficient; services can later be extracted into microservices (e.g. loan service, training service) as usage grows.

1.1 Component Layers

1. **Client Layer:** Web and mobile applications built using React/React Native (or Flutter). They communicate with the backend via REST/GraphQL APIs over HTTPS.
2. **API Gateway:** Single entry point for all client requests. Performs authentication, rate limiting and routing to internal services. Tools like **Kong** or **Express Gateway** can be used.
3. **Application Layer:** Contains business logic, validation, orchestration of workflows (loan application, disbursement, milestone verification). Implemented in a server-side framework.
4. **Data Layer:** Relational database (PostgreSQL) with support for complex queries and ACID transactions; caching layer (Redis) for sessions and frequently accessed data.
5. **Integration Layer:** Connects to external systems:
 - **Bank API** for loan application submission, disbursement and repayment events.
 - **UPI/NACH** payment gateway for AutoPay mandate and EMI deductions.
 - **DigiLocker** API for Aadhaar eKYC and certificate storage.
 - **TP Systems:** via secure API or file imports for attendance and assessment data.
6. **Event & Messaging:** Use a message broker (e.g. RabbitMQ, Apache Kafka) to decouple asynchronous tasks (e.g. sending notifications, updating analytics, triggering alerts) from synchronous API calls.

1.2 Technology Choices

Layer	Technology Recommendations	Rationale
Backend Framework	Python (Django/DRF) or Node.js (NestJS/Express + TypeScript)	Both support rapid development, strong community, and robust security libraries. Django provides batteries-included features (ORM, auth), whereas NestJS offers modular architecture and TypeScript type safety. Choose based on team expertise.
Database	PostgreSQL	ACID compliance, advanced indexing, JSONB support. Scale vertically and horizontally using read replicas.
Cache	Redis	In-memory storage for sessions, rate limiting, and caching course lists.
Message Broker	RabbitMQ or Apache Kafka	Enables asynchronous processing of events (notifications, analytics updates).
Search & Analytics	Elasticsearch (optional)	Full-text search on courses and logs, aggregated metrics for dashboards.
Authentication	JWT + OAuth 2.0	Secure stateless authentication; integration with Aadhaar eKYC and bank SSO if needed.
Hosting & Deployment	Kubernetes on cloud (e.g. AWS/GCP/Azure) or state data centre	Container orchestration ensures scalability; can deploy on-prem if required for data sovereignty.
Monitoring	Prometheus + Grafana, ELK Stack	Collect metrics, logs and set up alerting.
CI/CD	GitHub Actions or GitLab CI	Automate testing, building, security scanning and deployments.

2. Data Model & Schema

2.1 Entities

1. **User**
2. id (PK)
3. name
4. phone_number

- 5. email
- 6. password_hash
- 7. language_preference (enum: EN, HI, UR, KS)
- 8. role (enum: LEARNER, TP, BANK_USER, ADMIN)
- 9. aadhar_id (hashed)
- 10. borrower_score (numeric) – updated via analytics service
- 11. created_at, updated_at

12. TrainingProvider

- 13. id (PK)
- 14. name, registration_number
- 15. address, contact_email, contact_phone
- 16. tp_score (numeric)
- 17. guarantee_balance (decimal)
- 18. is_accredited (boolean)
- 19. created_at, updated_at

20. Course

- 21. id (PK)
- 22. training_provider_id (FK → TrainingProvider)
- 23. title, sector, nsqf_level, mode (enum: ONLINE, HYBRID, OFFLINE)
- 24. fee (decimal)
- 25. duration_weeks
- 26. language (JSON array – languages available)
- 27. curriculum (JSON – modules, assessments)
- 28. job_placement_rate (numeric)
- 29. guarantee_refund_policy (JSON – dropout_refund_pct, non_certification_refund_pct)
- 30. created_at, updated_at

31. Loan

32.id (PK)
33.user_id (FK → User)
34.course_id (FK → Course)
35.bank_id (FK → Bank – or store within Loan for pilot)
36.principal_amount (decimal)
37.interest_rate (decimal)
38.tenure_months
39.moratorium_end_date
40.status (enum: PENDING, APPROVED, ACTIVE, COMPLETED, DEFAULTED, REJECTED)
41.borrower_score_at_approval
42.tp_score_at_approval
43.guarantee_cover_pct (decimal)
44.created_at, updated_at

45. EscrowLedger

46.id (PK)
47.loan_id (FK → Loan)
48.tp_id (FK → TrainingProvider)
49.balance (decimal)
50.created_at, updated_at

51. Disbursement

52.id (PK)
53.loan_id (FK → Loan)
54.milestone (enum: ENROLMENT, PROGRESS, ASSESSMENT, CERTIFICATION)
55.amount (decimal)
56.status (enum: SCHEDULED, RELEASED, WITHHELD)
57.scheduled_date, released_date

58. Repayment

59.id (PK)

60.loan_id (FK → Loan)

61.due_date

62.amount

63.status (enum: UPCOMING, PAID, FAILED, DEFERRED)

64.payment_reference (string – UPI mandate ID / transaction ID)

65.created_at, updated_at

66.Attendance

67.id (PK)

68.loan_id (FK → Loan)

69.date

70.mode (enum: ONLINE, OFFLINE)

71.hours_attended

72.verified_by (FK → TrainingProvider user or facilitator)

73.created_at

74.Assessment

75.id (PK)

76.loan_id (FK → Loan)

77.module

78.date

79.score

80.max_score

81.passed (boolean)

82.verified_by (FK → TrainingProvider)

83.created_at

84.Bank

- id (PK)

- name

- api_endpoint
- priority_sector_report_endpoint
- created_at, updated_at

85. Notification

- id (PK)
- user_id (FK → User)
- type (enum: INFO, WARNING, ALERT)
- message
- is_read (boolean)
- created_at

2.2 Relationships

- A **User** may have multiple **Loans**; a loan belongs to one user.
- A **TrainingProvider** can offer multiple **Courses**; a course belongs to one provider.
- **Loans** link **Users**, **Courses**, **TrainingProviders**, **Banks** and **EscrowLedgers**.
- **Disbursements** and **Repayments** track money movements for a loan; they reference the loan and milestone.
- **Attendance** and **Assessments** are tied to a loan (because they relate to a particular learner's instance of a course) and are verified by training providers.

3. API Design

The platform exposes a set of RESTful endpoints under `/api/v1/`. For future flexibility, GraphQL can be introduced; however, REST is widely supported and simpler for bank integrations.

3.1 Authentication & Authorization

- POST `/auth/register` – Create new user. Accepts phone, password, language, role.
- POST `/auth/login` – Authenticate with phone/email and password, returns JWT.
- POST `/auth/refresh` – Obtain new JWT.

Role-based access control is enforced via middleware; tokens include roles and permissions.

3.2 Course & TP Management

- GET /courses – List courses; query params for sector, mode, price range, language.
- GET /courses/{id} – Get course details.
- POST /tp/register – Training provider registration; admin approval required.
- POST /tp/{id}/courses – Create course (TP only).
- GET /tp/{id}/courses – List TP's courses.
- PATCH /courses/{id} – Update course (TP or admin).

3.3 Loan Workflow

- POST /loans/apply – Learner applies for a loan with course_id and personal data. Backend calculates scores and sends to bank API.
- GET /loans/{id} – Retrieve loan status and details.
- POST /loans/{id}/accept – Learner accepts approved loan and signs; triggers first disbursement.
- POST /loans/{id}/reject – Learner declines offer.
- GET /loans/{id}/repayments – List repayment schedule.

3.4 Disbursement & Milestones

- POST /loans/{id}/milestones/{milestone}/verify – TP submits evidence for milestone; includes evidence documents/URLs.
- POST /loans/{id}/disburse/{milestone} – System or bank triggers disbursement after verification.

3.5 Attendance & Assessment

- POST /loans/{id}/attendance – Submit attendance record (TP or facilitator).
- POST /loans/{id}/assessment – Submit assessment result.

3.6 Payment Integration

- POST /loans/{id}/setup-mandate – Initiate UPI/NACH e-mandate creation with payment gateway. Returns mandate ID.
- POST /payments/callback – Payment gateway callback for EMI deduction events; updates Repayment status.

3.7 Notifications & Monitoring

- GET /notifications – User fetches notifications.
- POST /notifications – System creates notifications (for asynchronous tasks).

3.8 Admin APIs

- GET /admin/overview – Returns aggregated metrics.
- POST /admin/settings – Update policy parameters (guarantee percentages, threshold values).
- GET /admin/tp-requests – List pending training providers.
- PATCH /admin/tp/{id} – Approve or reject TP.

3.9 Bank Integration APIs

Banks integrate using secure endpoints (e.g. <https://bankapi.example.com/skill-loans>). Key operations include:

- POST /applications – Receive loan application data; respond with decision and terms.
- POST /disbursements – Bank receives request to disburse tranche; returns transaction ID.
- POST /repayments – Bank sends repayment events (paid/failed), NACH/UPI status.
- GET /reports/psl – Bank extracts aggregated data for PSL compliance[1].

4. Integration & Workflow Orchestration

1. **Loan application submission** triggers an **ApplicationSubmitted** event. A background worker packages the data and calls the bank API. The response is stored; if approved, a **LoanApproved** event is emitted.
2. **Loan approval** leads to creation of an **EscrowLedger** and **Disbursement** records. The system schedules the first disbursement. **PaymentGatewayService** sets up UPI AutoPay.
3. **Milestone verification** results in **MilestoneVerified** events, which trigger disbursement workflows; if milestone is not met, the **Disbursement** remains withheld and a **MilestoneFailed** event triggers learner notifications.
4. **Repayment failure** triggers a **PaymentFailed** event; if multiple failures occur, a **Default** event triggers TP guarantee deductions and escalations.

5. Security Considerations

See separate Security Checklist document for detailed controls. In the backend design:

- Use TLS for all network communication. Terminate TLS at the API gateway.
- Store sensitive data (Aadhaar numbers, KYC documents) encrypted using field-level encryption. Do not log PII.
- Implement rate limiting and input validation to prevent injection attacks. Use ORM parameterization for DB queries.
- Conduct regular vulnerability scans and maintain a secure secrets management system (e.g. HashiCorp Vault).
- Follow the principle of least privilege: separate roles and restrict DB access accordingly.

6. Data Privacy & Compliance

- Collect only necessary personal data. Provide explicit consent language in all forms.
- Facilitate user access to their data and ability to request deletion (subject to financial records retention laws).
- Comply with the RBI's master direction on digital lending, the IT Act and any state-specific data regulations.

7. Future Scalability & Microservices

As transaction volume grows, the monolithic service can be broken into domain-specific services (e.g. **Loan Service**, **Course Service**, **User Service**, **Analytics Service**) communicating via APIs or message queues. A service mesh (e.g. Istio) can manage inter-service communication and security. Multi-region deployment may be supported to ensure low latency and resilience against local outages.

[1] Priority Sector Lending In India: A Guide

<https://www.go-yubi.com/securitisation/priority-sector-lending/>